AI 法律文书 Agent 项目报告

一、 问题与用户

- **目标用户**：处理日常民事纠纷的初级律师、法务人员，以及遇到租房纠纷需要初步法律建议的普通当事人。
- **真实场景**：房屋租赁纠纷（如押金退还、房屋损坏、提前退租）。
- **现有困难**：案件事实通常分散在聊天记录和当事人零散的陈述中，人工提取费时；检索适用法条与相关类案的前期准备工作繁重，导致出具初步诉讼方案的效率低下。
- **最终结果与成功标准**：用户输入案情和诉求后，系统能在 10 秒内自动输出结构化的相关法条、案件争点（有利/不利分析）以及一份可继续编辑的诉讼方案初稿。
- **聚焦范围原因**：第一版 MVP 聚焦于“房屋租赁纠纷”这一细分场景，是为了在有限的时间内验证“本地知识库检索+大模型生成”核心工作流的切实可行性，避免因法律领域过宽导致数据准备和模型调优失去焦点。

二、 方案与交互

- **核心任务流程**：输入案情与诉求 -> 触发后端检索本地法律库 -> 组装 Prompt 提交大模型 -> 结构化数据返回前端渲染。
- **页面配合**：采用左右分屏布局。左侧为**对话/输入区**，用户提交案件事实与诉求；右侧为**工作区**，默认隐藏，请求成功后展示结构化的法律依据、争点卡片和文书初稿。
- **关键与异常状态**：
  - **加载状态**：点击生成后，按钮锁定并显示“加载中”，防止重复提交。
  - **异常状态**：若大模型 API 调用失败（如余额不足触发 HTTP 402 错误），界面会捕获错误并提示具体异常原因，防止页面卡死。
- **高价值交互设计**：右侧的“诉讼方案卡片”被设计为一个**可编辑的文本区域（`<textarea>`）**。这一设计极大提升了实用性，允许律师在 AI 生成的初稿基础上直接进行人为修正和补充，体现了“AI 只作辅助，决策由人负责”的理念。

三、 实现与技术

- **前端**：原生 HTML + JavaScript (Fetch API)，搭配 Tailwind CSS CDN 进行快速响应式样式构建，未使用复杂的前端框架，实现轻量化。
- **后端**：Python + FastAPI，搭配 Uvicorn 提供接口服务。
- **数据**：使用轻量级本地 JSON 文件（`laws_db.json`）模拟向量知识库，存储《民法典》租赁合同相关真实法条与类案。
- **模型**：接入 DeepSeek 大语言模型（通过 OpenAI SDK 兼容接口），负责文本理解与结构化 JSON 生成。
- **部署**：前端与后端一并挂载部署于 Vercel 云平台，通过 `vercel.json` 配置文件实现无服务器 (Serverless) 路由转发。

四、 完成范围与限制

- **已真实实现并运行**：前端分屏交互逻辑、后端 FastAPI 接口、本地 JSON 检索机制、真实的大模型 API 接入与 Prompt 组装、基于 Vercel 的公网在线部署。
- **部分实现**：法律资料库目前仅涵盖房屋租赁纠纷的部分精选法条，未接入国家裁判文书网等实时外部 API。
- **尚未实现（受限原因及后续计划）**：
  - 未实现用户登录与历史记录保存（受限于开发时间，计划后续引入数据库实现）。
  - 未实现真正的向量检索（受限于算力资源，当前采用简易本地匹配，计划后续引入 Chroma 或 Qdrant）。

五、 测试记录

- **完整流程测试**：
  - **输入**：承租人提前退租，房东以家具损坏为由拒退押金。
  - **结果**：系统成功引用《民法典》违约责任条款，在争点卡片中准确指出了"正常损耗无需赔偿"的有利依据和"提前退租构成违约"的不利依据，并生成了起诉状初稿。
- **常见失败场景测试**：
  - **场景**：API 账户余额耗尽（返回 402 错误）。
  - **发现问题与修改**：最初页面会因无数据返回而崩溃。随后在后端加入了 `try-except` 异常捕获机制，在前端加入了错误信息渲染逻辑。修改后，若遇 API 异常，页面底部会打印清晰的 Error Code，提示用户检查模型服务。

六、 AI 协作与个人完成内容

- **AI 协作部分**：使用 AI 编程助手 (Trae) 辅助生成了前后端的基础框架代码（如 FastAPI 的路由定义、HTML 布局代码）、项目所需的环境配置文件（`requirements.txt`，`vercel.json`），以及项目初版 README 文档。
- **个人完成内容**：作为独立开发者，个人完成了整体产品逻辑设计、技术栈选型（FastAPI + Vercel）、法律资料库（`laws_db.json`）的真实数据搜集与清洗、核心 Prompt 的调试与优化、前后端联调测试，以及最终的云端部署与异常排查工作。确保了生成的代码不仅能跑通，而且真正解决了法律文书初具的业务痛点。

AI Legal Document Agent Project Report

1. Problem & User

- **Target Users**: Junior lawyers, legal assistants handling daily civil disputes, and ordinary individuals seeking preliminary legal advice for rental conflicts.
- **Real Scenario**: Housing rental disputes (e.g., deposit refunds, property damage, early termination).

- **Current Difficulties**: Case facts are often scattered across chat logs and fragmented client statements, making manual extraction time-consuming. Researching applicable laws and precedent cases requires heavy preparatory work, reducing the efficiency of drafting initial litigation plans.
- **Final Result & Success Criteria**: After inputting facts and claims, the system automatically outputs structured relevant laws, case issues (pro/con analysis), and an editable draft litigation plan within 10 seconds.
- **Reason for Scope Focus**: The MVP strictly focuses on the "housing rental dispute" niche to validate the core workflow ("local knowledge base retrieval + LLM generation") within a limited timeframe, avoiding the loss of focus on data preparation and model tuning caused by an overly broad legal scope.

2. Solution & Interaction

- **Core Task Flow**: Input facts & claims -> Trigger backend to retrieve local legal DB -> Assemble Prompt for LLM -> Return structured data for frontend rendering.
- **UI Coordination**: A split-screen layout is adopted. The left side is the **Input Area** for users to submit facts and claims; the right side is the **Workspace** (hidden by default), which displays structured legal bases, issue cards, and document drafts upon a successful request.
- **Key & Abnormal States**:
  - **Loading State**: Upon clicking "Generate", the button locks and displays a loading indicator to prevent duplicate submissions.
  - **Abnormal State**: If the LLM API call fails (e.g., HTTP 402 due to insufficient balance), the UI catches the error and displays the specific cause, preventing the page from freezing.
- **High-Value Interaction Design**: The "Litigation Plan Card" on the right is designed as an **editable text area (`<textarea>`)**. This significantly enhances practicality, allowing lawyers to directly modify and supplement the AI-generated draft, embodying the philosophy that "AI acts as an assistant, while humans make the final decisions."

3. Implementation & Tech

- **Frontend**: Native HTML + JavaScript (Fetch API), styled with Tailwind CSS CDN for quick responsive design. No complex frontend frameworks were used to keep it lightweight.
- **Backend**: Python + FastAPI, served via Uvicorn.
- **Data**: A lightweight local JSON file (`laws_db.json`) simulates a vector knowledge base, storing authentic articles from the Civil Code and precedent summaries related to rental contracts.
- **Model**: Integrated with the DeepSeek Large Language Model (via OpenAI SDK compatible interface) for text comprehension and structured JSON generation.

- **Deployment**: Both frontend and backend are deployed on the Vercel cloud platform, utilizing a `vercel.json` configuration file for Serverless routing.

4. Scope & Limitations

- **Fully Realized & Operational**: Frontend split-screen interaction logic, backend FastAPI endpoints, local JSON retrieval mechanism, authentic LLM API integration & Prompt assembly, and public online deployment via Vercel.
- **Partially Realized**: The legal database currently only covers selected laws specific to housing rental disputes and is not connected to real-time external APIs like the National Judgement Document public database.
- **Unrealized (Limitations & Future Plans)**:
  - User login and history saving (limited by development time; planned to introduce a database later).
  - True vector retrieval (limited by computing resources; currently using simple local keyword matching; planned to introduce Chroma or Qdrant later).

5. Test Records

- **Complete Flow Test**:
  - **Input**: Tenant terminates lease early; landlord refuses to refund deposit citing damaged furniture.
  - **Result**: The system successfully cited the Civil Code's breach of contract clauses, accurately identified the favorable basis ("normal wear and tear requires no compensation") and unfavorable basis ("early termination constitutes a breach") in the issue card, and generated a draft complaint.
- **Common Failure Scenario Test**:
  - **Scenario**: API account balance depleted (Returns 402 Error).
  - **Issues Found & Fixes**: Initially, the page crashed due to the lack of returned data. A `try-except` error-catching mechanism was subsequently added to the backend, along with error rendering logic on the frontend. After the fix, encountering an API error prints a clear Error Code at the bottom of the page, prompting the user to check the model service.

6. AI Collaboration & Individual Contributions

- **AI Collaboration**: Used an AI programming assistant (Trae) to help generate foundational boilerplate code (e.g., FastAPI routing, HTML layout), environment configuration files (`requirements.txt` , `vercel.json`), and the initial project README documentation.
- **Individual Contributions**: As a solo developer, I completed the overall product logic design, tech stack selection (FastAPI + Vercel), gathering and cleaning of authentic data for the legal database (`laws_db.json`), debugging

and optimization of the core Prompts, frontend-backend integration testing, and the final cloud deployment and troubleshooting. This ensured the generated code was not only functional but effectively solved real business pain points in legal document drafting.